



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BENGALURU – 560100, KARNATAKA, INDIA

B.TECH. (CSE)
V SEMESTER
UE24CS341A – SOFTWARE ENGINEERING

HIGH LEVEL SOFTWARE ARCHITECTURE DOCUMENT

ON

**UpSkill-AI: An AI-Powered Upskilling Recommendation
and Automation System for Students**

Design Phase Deliverable

TEAM NUMBER: 18 PROJECT ID: 22

SUBMITTED BY

Name	SRN	Module Owned
VANSH SHARMA	PES2UG24AM179	M1 – Profile & Skill Extraction
YAJAT SONI	PES2UG24AM185	M2 – Gap Analysis & Roadmap
HEMANTH KUMAR R	PES2UG24AM202	M3 – Progress & Adaptive Adjustment
SHUBHIKA PRADEEP	PES2UG24AM157	M4 – Catalogue, Auth & RBAC

August – November 2026

Revision History

Name	Date	Reason For Changes	Version
VANSH SHARMA, YAJAT SONI, HEMANTH KUMAR R, SHUBHIKA PRADEEP	30-09-2026	Initial version	1.0

1. Introduction

1.1 Purpose

This document describes the high level software architecture of UpSkill-AI, an upskilling recommendation and progress tracking system for students. It defines how the system is decomposed into modules, how those modules interact, what interfaces they expose, how data is organised, and how the system is deployed.

The architecture described here realises the requirements specified in the UpSkill-AI Software Requirements Specification, version 1.0. Where the SRS states what the system must do, this document states how the system is structured in order to do it. Detailed class-level and method-level design is deferred to the Software Design Document.

1.2 Scope

This document covers the complete UpSkill-AI system, comprising four functional modules:

- M1 — Student Profile and Skill Extraction
- M2 — Skill Gap Analysis and Personalized Roadmap Generation
- M3 — Progress Tracking and Adaptive Adjustment
- M4 — Resource Catalogue, Authentication and Role-Based Access Control

Each module is owned end-to-end by one team member, which is why module boundaries are drawn along functional lines rather than along technical layers.

1.3 Definitions and Abbreviations

Term	Meaning
ADR	Architecture Decision Record — a recorded decision with its context and consequences
Canonical skill	The single agreed name for a skill in the taxonomy; all variant spellings resolve to it
JWT	JSON Web Token — the token format used for authenticated sessions
ORM	Object Relational Mapper — SQLAlchemy, used in the data access layer
RBAC	Role-Based Access Control
Skill gap	A skill required by a target role that the student does not hold, or holds below the required level
Skill vector	The structured set of canonical skills a student holds, with the level held for each
Stale recommendation	A recommendation generated before a change to the skill vector or target role

1.4 References

Ref	Document	Version
R1	UpSkill-AI Software Requirements Specification	1.0
R2	Jackfruit Software Engineering Mini-Project Guidelines 2026, UE24CS341A	2026
R3	UpSkill-AI Project Test Plan (Team 18)	1.0
R4	UpSkill-AI Product Backlog (Team 18)	1.0
R5	UpSkill-AI Software Design Document	To be produced

2. Architectural Goals and Constraints

2.1 Architectural Goals

The architecture is shaped by five goals, listed in priority order. Where two goals conflict, the higher one wins.

#	Goal	Why it drives the architecture
G1	Independent module ownership	Each of the four members must complete one full functionality and be individually assessed. Module boundaries must therefore be clean enough that one member's work can progress and be reviewed without waiting on another's internals.
G2	Testability of core logic	Gap computation, prerequisite sequencing and permission checks must be unit-testable with fixed inputs and no external dependencies, because unit test coverage is separately assessed.
G3	Explainability of recommendations	Every recommendation must be traceable to the specific skill gap it closes. This rules out an opaque scoring function and requires the gap result to be carried through to the output.
G4	Graceful degradation	A failure or quota exhaustion in the external resource service must not break roadmap generation. The catalogue, not the external API, is the system of record.
G5	Reproducible builds and deployment	The full stack must start from one compose definition so that the CI pipeline, each developer machine and the demonstration environment behave identically.

2.2 Constraints

#	Constraint	Source
C1	Frontend in Next.js (React); backend in Python with FastAPI; persistence in PostgreSQL. All modules implemented within this stack.	Team decision, SRS §2.5
C2	Version control on GitHub with pull-request review; CI/CD through Jenkins; containerisation with Docker; unit testing with pytest; static analysis with SonarQube.	Course-mandated process, R2
C3	Recommendation logic in Phase 1 is a deterministic rule-based, level-aware matcher rather than a trained model.	SRS §2.5
C4	YouTube Data API v3 free tier has a daily query quota; external results must be cached and reused per skill.	SRS §2.5, §2.6
C5	Role-based access must be enforced server-side; hiding a UI control does not constitute access control.	SRS §5.4.4
C6	Single-semester delivery by a four-member team with no dedicated operations support.	Project timeline

2.3 Consequences of the Constraints

Constraint C6 is the one that most shapes the result. A four-member team delivering in one semester cannot operate a distributed system, so the architecture deliberately avoids network boundaries between the four modules. Constraint C1 fixes a single backend runtime, which means the modules share a process and can call each other directly. Goal G1 then has to be achieved by discipline at the interface level rather than by physical separation, which is why §4.3 states the module interaction rules explicitly.

3. Architectural Style and Justification

3.1 Chosen Style: Layered Modular Monolith

UpSkill-AI is built as a layered modular monolith. The backend runs as a single deployable service, internally divided into five strict layers, with the business logic layer further divided into four modules that align one-to-one with the four owned functionalities.

Two structuring principles apply simultaneously:

- Horizontal layering — presentation, API, business logic, data access, persistence. Each layer may call only the layer directly beneath it. Upward calls and skip-layer calls are prohibited.
- Vertical modularisation — within the business logic layer, the four modules communicate only through each other's published service interfaces, never by reaching into another module's repositories or internal helpers.

The horizontal rule keeps technical concerns separated and makes the data access layer swappable.

The vertical rule delivers goal G1: a member can change the internals of their module freely as long as the published interface holds.

3.2 Alternatives Considered and Rejected

Alternative	Why it was rejected
Microservices — each of the four modules as an independently deployed service	Superficially attractive because it maps to the four owners, but it converts every internal call into a network call, and would require service discovery, inter-service authentication, distributed transactions across skill-vector updates, and four separate CI pipelines. For a four-member team in one semester this is operational overhead with no offsetting benefit at the expected user load. The same ownership separation is achievable through module interface discipline at a fraction of the cost.
Single-layer MVC with business logic in controllers	Simplest to write initially, but it puts gap computation and permission checks inside HTTP request handlers. Those then need a simulated HTTP request to test, which directly undermines goal G2 and the assessed unit-test coverage. Rejected on testability grounds.
Event-driven architecture with a message broker	A reasonable fit for the progress-to-re-rank flow, which is genuinely event-shaped. Rejected because introducing a broker adds a further container, failure mode and debugging surface to support exactly one asynchronous interaction. The same behaviour is achieved with a synchronous in-process call from M3, and the design leaves room to move to a broker later if the interaction count grows.
Serverless functions	Incompatible with constraint C2, which mandates Docker containerisation and a Jenkins deployment pipeline. Cold-start latency would also work against NFR-PERF-03.

3.3 Known Trade-offs of the Chosen Style

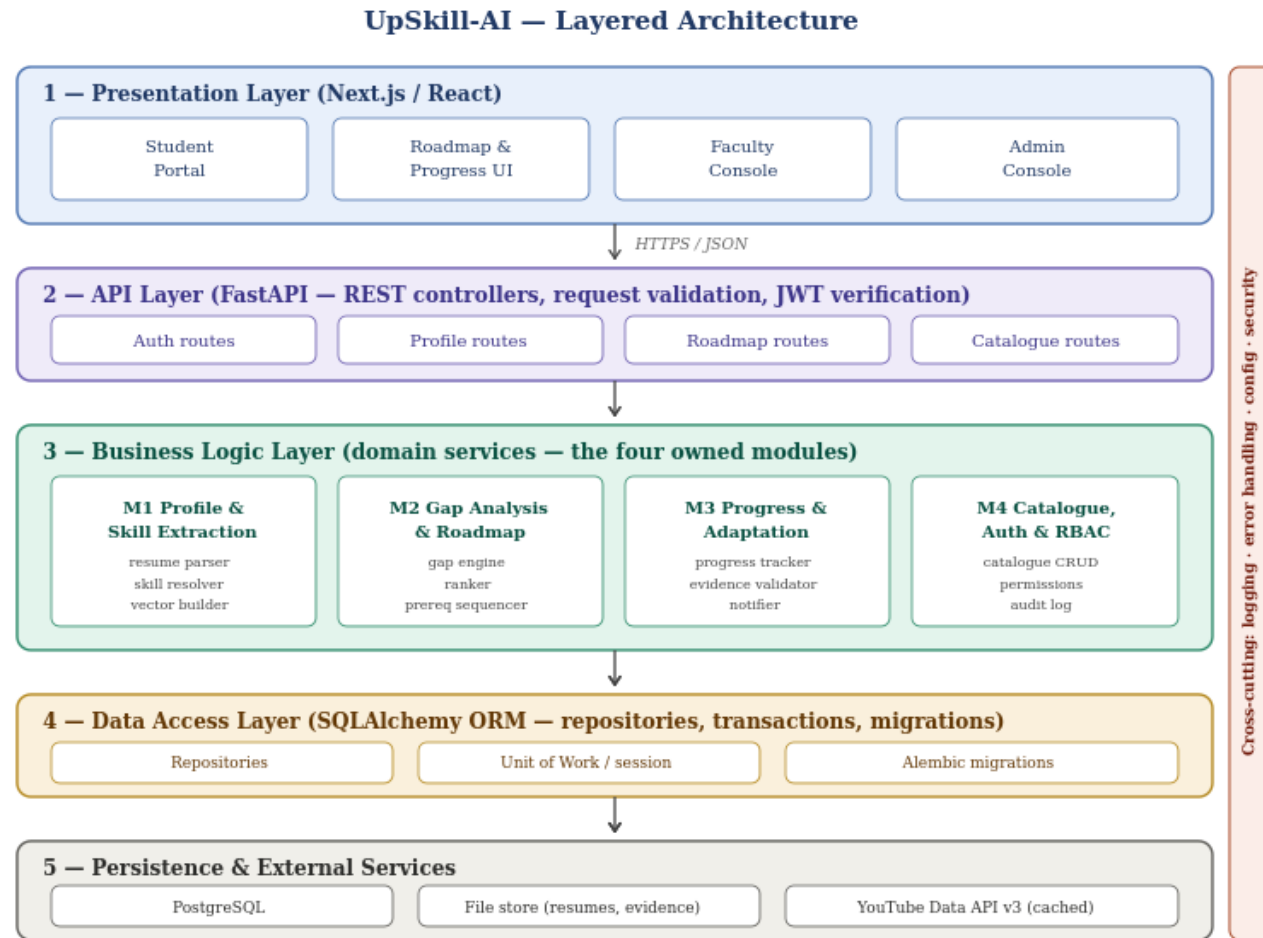
A modular monolith is honest about its own limits, and these should be stated rather than discovered later:

- Modules cannot be scaled independently. If roadmap generation becomes the bottleneck, the whole backend must be scaled. This is acceptable at the expected cohort-sized load and is explicitly a future concern.
- Module boundaries are enforced by convention and code review, not by the compiler or the network. A careless import across module internals will work at runtime. This is mitigated by directory structure, a documented import rule, and review on every pull request.
- A crash in one module takes the whole backend down, unlike a service boundary. Mitigated by error isolation at the API layer, so a failure inside one route does not propagate.

4. Architectural Views

4.1 Logical View — Layer Decomposition

The diagram below shows the five layers and the components within each. The cross-cutting concerns shown on the right — logging, error handling, configuration and security — are implemented as shared middleware and utilities used by every layer rather than as a layer of their own.



Each layer depends only on the layer directly below it. No upward calls and no skip-layer calls are permitted.

Figure 1 — Layered architecture of UpSkill-AI

4.2 Module View — Component Interaction

This view shows the four business-logic modules, the public operations each exposes, and the permitted interactions between them. The skill taxonomy sits at the centre as shared read-only reference data: every module reads it, no module owns it at runtime, and it is maintained by administrators through M4.

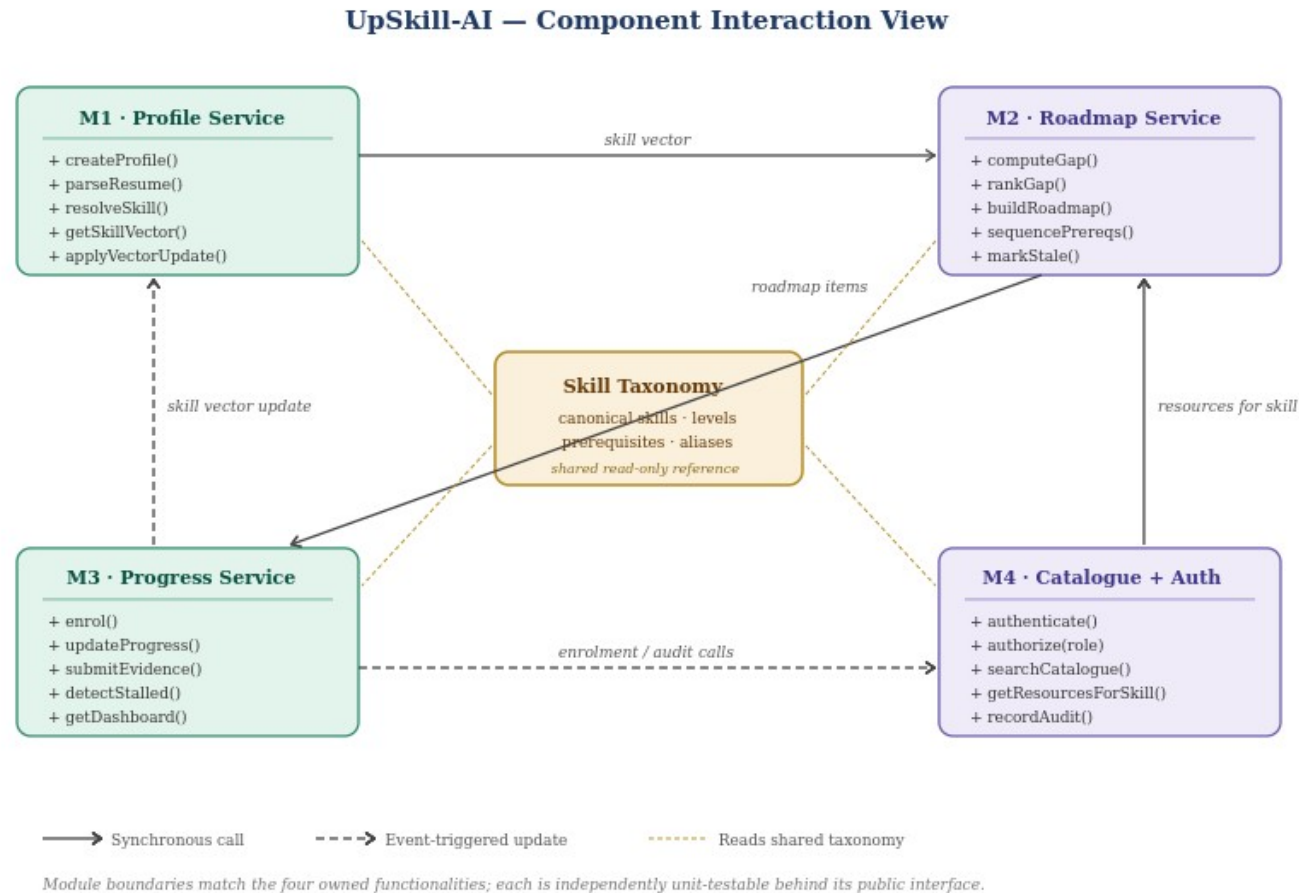


Figure 2 — Component interaction view

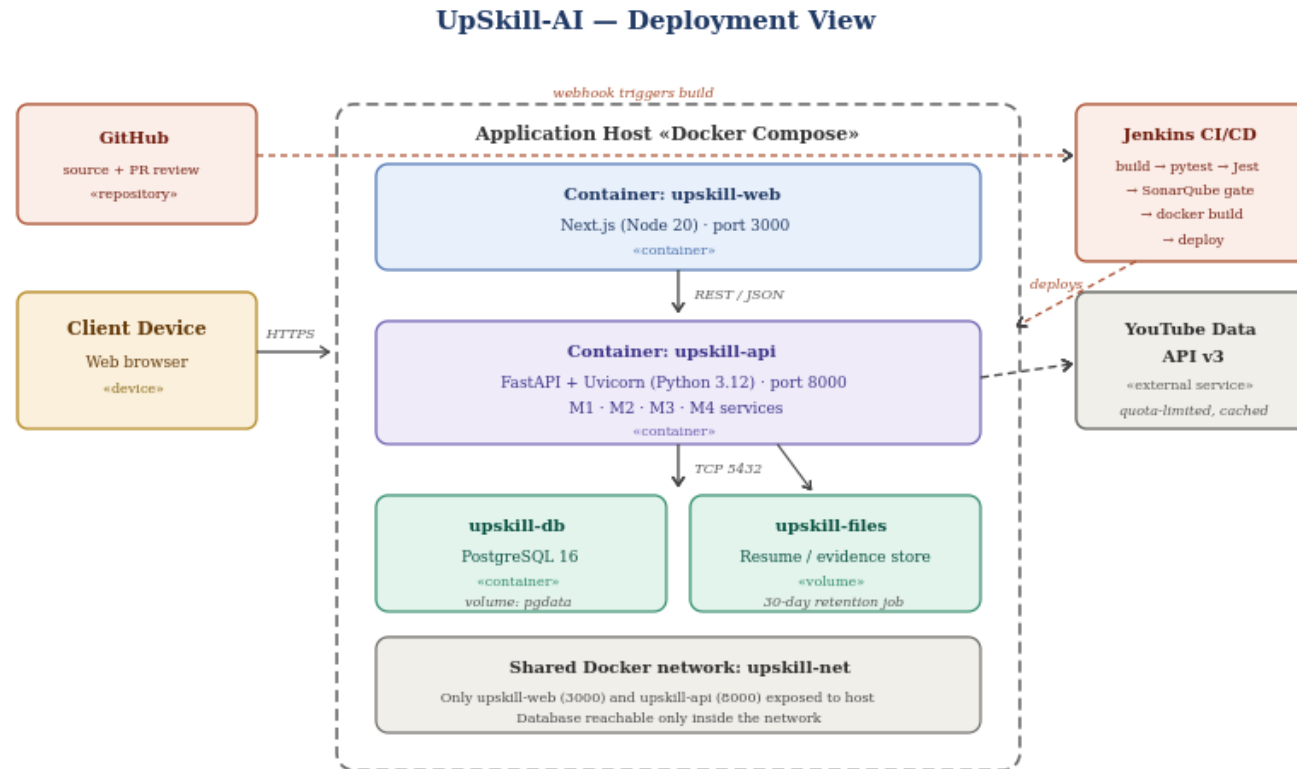
4.3 Module Interaction Rules

These rules are what make goal G1 real. They are enforced in code review on every pull request.

#	Rule	Rationale
R1	A module may call another module only through that module's published service interface (the operations listed in Figure 2).	Keeps each member free to change internals without breaking another member's work.
R2	A module may not import another module's repository, model or internal helper directly.	Prevents hidden coupling that survives interface changes.
R3	The skill vector format and the skill taxonomy schema are frozen shared contracts. Changing either requires agreement from all four owners.	These two structures are read by every module; unilateral change breaks the system.
R4	M1 is the only module permitted to write a student's skill vector. M3 requests a change; it does not perform one.	Single writer prevents conflicting concurrent updates to the same derived data.
R5	M4 is the only module permitted to write catalogue and user records. Other modules read through its interface.	Same single-writer principle applied to catalogue and identity data.
R6	No module calls the external YouTube Data API directly except M4, which owns the cache and the quota budget.	Centralises quota management and makes caching effective.

4.4 Deployment View

The system runs as three containers and one persistent volume on a shared Docker network, defined in a single docker-compose.yml. Only the web and API containers are exposed to the host; the database is reachable only from inside the network, which satisfies NFR-SEC-01 and NFR-SEC-02 at the infrastructure level as well as in application code.



The three runtime containers and the persistent volume are defined in a single docker-compose.yml, so the full stack is reproducible with one command. Development, CI and demonstration environments use the same compose definition with environment-specific variable files.

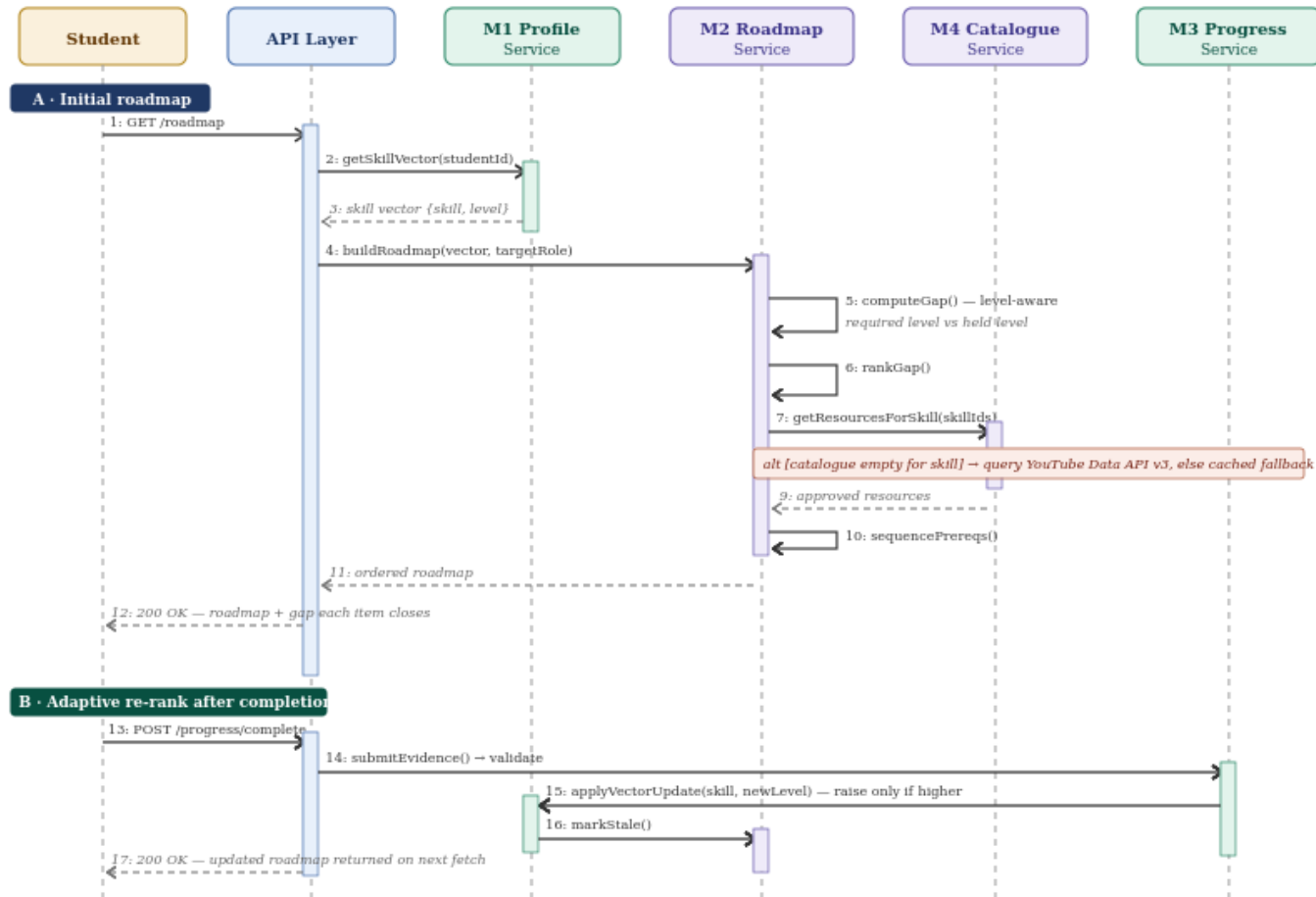
Figure 3 — Deployment view including CI/CD pipeline

4.5 Process View — Key Runtime Flow

The sequence below traces the system's central use case: generating a roadmap, and then adapting it when the student completes a learning resource. Part A is the read path. Part B is the adaptive path, which is the interaction that makes the system more than a static recommender.

Two design points are visible in this sequence. First, the alt fragment at step 8 shows graceful degradation (goal G4): an empty catalogue for a skill falls through to the external API or a cached list rather than surfacing an error. Second, in part B the re-rank is triggered by the validated evidence event itself, not deferred to the student's next login, so the roadmap the student sees is never stale relative to work they have just completed.

Sequence — Roadmap Generation and Adaptive Re-ranking



Re-ranking is event-triggered by a validated skill-vector change, not deferred to next login. Steps 5, 6 and 10 are deterministic and unit-testable in isolation.

Figure 4 — Sequence: roadmap generation and adaptive re-ranking

4.6 Data View

The entity relationship diagram below shows the persistent data model. STUDENT, SKILL and CAREER_ROLE form the core; the HAS_SKILL relationship carries the proficiency level and confidence score that together constitute the stored skill vector, and the REQUIRES relationship carries the required level that makes gap computation level-aware rather than a plain set difference.

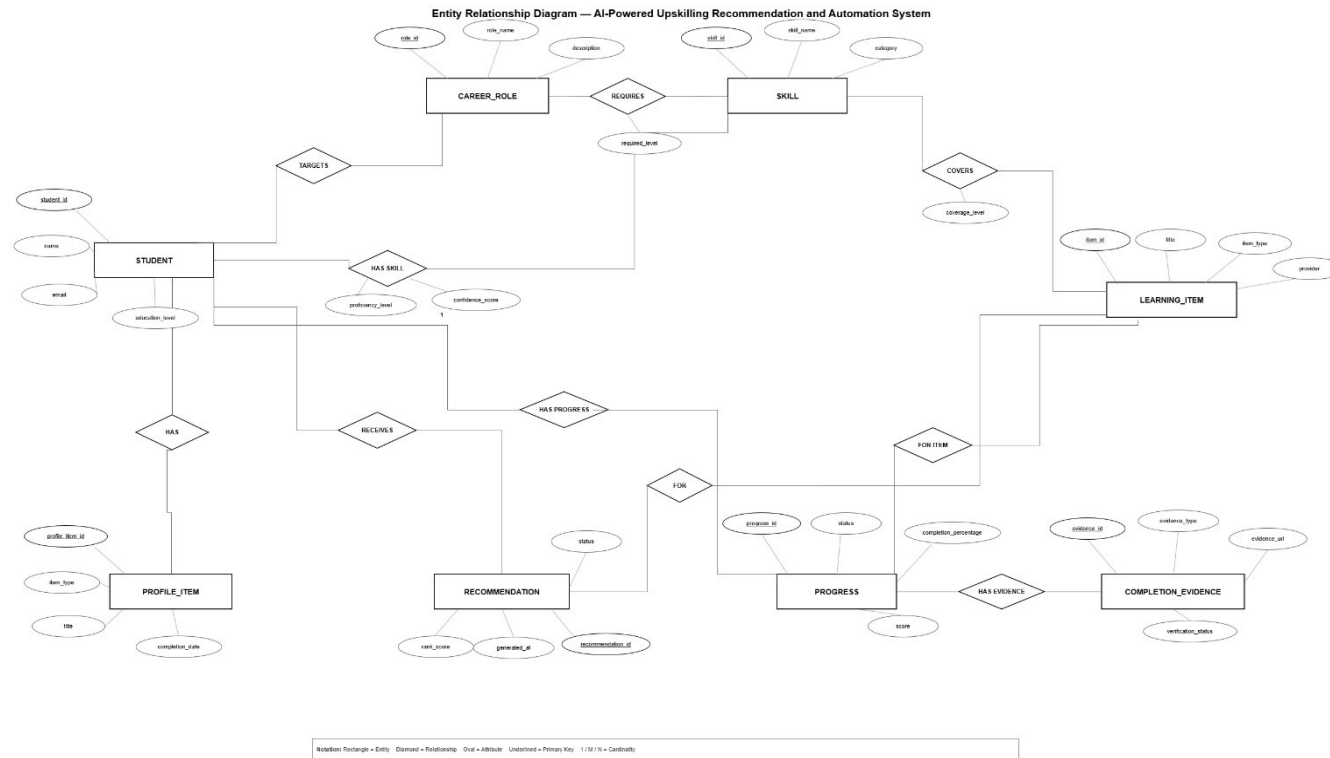


Figure 5 — Entity relationship diagram

Access control, audit and notification entities are owned by M4 and M3 respectively and are to be reflected in the final revision of Figure 5. The tables required are listed in §4.7.

4.7 Principal Data Stores by Module

Owning module (owner)	Tables / entities written	Read by
M1 — Profile (Vansh Sharma)	student, profile_item, student_skill (skill vector), skill_change_history, resume_upload	M2 (skill vector), M3 (vector update target), M4 (faculty read-only view)
M2 — Roadmap (Yajat Soni)	recommendation, gap_snapshot, recommendation_explanation	M3 (enrolment source), M4 (faculty endorse/override target)
M3 — Progress (Hemanth Kumar R)	progress, completion_evidence, milestone, notification	M2 (staleness trigger), M4 (faculty progress view and analytics)
M4 — Catalogue & Auth (Shubhika Pradeep)	learning_item, skill, skill_alias, skill_prerequisite, career_role, role_skill_requirement, user_account, role, mentor_assignment, audit_record	All modules (taxonomy and catalogue are read by everyone)

The single-writer rule (R4, R5) is visible in this table: no table appears in more than one row of the first column. This is the property that keeps concurrent module development from producing write conflicts.

5. Interface Definitions

5.1 REST API Surface

All endpoints are prefixed /api/v1. Every endpoint except registration and login requires a valid JWT. The Role column states the minimum role required; authorisation is checked server-side in every case.

Method & Path	Module	Purpose	Role	Requirement
POST /auth/register	M4	Create a user account	Public	REQ-F4-01
POST /auth/login	M4	Authenticate and issue JWT	Public	REQ-F4-01
GET /profile	M1	Retrieve the caller's profile	Student	REQ-F1-01, F1-39
POST /profile/courses	M1	Add a course entry	Student	REQ-F1-02
POST /profile/projects	M1	Add a project entry	Student	REQ-F1-03
POST /profile/skills	M1	Manually add a skill	Student	REQ-F1-04, F1-05
POST /profile/resume	M1	Upload a resume for extraction	Student	REQ-F1-08 to F1-11
GET /profile/resume/extracted	M1	List extracted skills pending confirmation	Student	REQ-F1-13, F1-14
POST /profile/skills/confirm	M1	Confirm or reject extracted skills	Student	REQ-F1-15 to F1-17
GET /profile/skill-vector	M1	Return the current skill vector	Student	REQ-F1-25, F1-27
PUT /profile/target-role	M1	Select or change the target role	Student	REQ-F1-31 to F1-33
GET /roadmap	M2	Return the ranked, sequenced roadmap	Student	REQ-F2-05, F2-06
GET /roadmap/gap	M2	Return the computed level-aware skill gap	Student	REQ-F2-02 to F2-04
POST /roadmap/recompute	M2	Force recomputation of gap and roadmap	Student	REQ-F2-08
POST /progress/enrol	M3	Enrol in a recommended resource	Student	REQ-F3-01
PATCH /progress/{id}	M3	Update completion percentage or state	Student	REQ-F3-02, F3-03
POST /progress/{id}/complete	M3	Mark complete and submit evidence	Student	REQ-F3-04 to F3-07
GET /progress/dashboard	M3	Progress over time, gaps closed, streaks	Student	REQ-F3-10 to F3-12
GET /catalogue	M4	Search and filter learning resources	Student	REQ-F4-09, F4-10, F4-12
POST /catalogue	M4	Create a catalogue resource	Administrator	REQ-F4-05 to F4-08
PATCH /catalogue/{id}/moderate	M4	Approve, reject or deactivate a resource	Administrator	REQ-F4-11
GET /faculty/mentees	M4	List assigned mentees and their progress	Faculty-Mentor	REQ-F4-13
POST /faculty/recommendation/{id}/endorse	M4	Endorse a recommendation	Faculty-Mentor	REQ-F4-14
POST /faculty/recommendation/{id}/override	M4	Override a recommendation with a reason	Faculty-Mentor	REQ-F4-15
GET /admin/users	M4	List and manage user accounts and roles	Administrator	REQ-F4-02
GET /admin/unresolved-skills	M4	Review unresolved skill strings	Administrator	REQ-F1-23

5.2 Internal Module Interfaces

These are in-process Python interfaces, not HTTP endpoints. They are the contracts referenced by rule R1.

Interface operation	Provider	Consumers	Contract summary
get_skill_vector(student_id) → SkillVector	M1	M2	Returns the list of canonical skills with level, confidence and source, in the Appendix B format. Never returns partial data; raises if the profile does not exist.
apply_vector_update(student_id, skill_id, level, source) → SkillVector	M1	M3	Raises a skill's level only if the incoming level is higher than the level currently held. Idempotent: a repeat call with the same or lower level is a no-op.
compute_gap(vector, role_id) → GapResult	M2	M1 (display), M3	Pure function over its inputs. No database access, no external calls. Deterministic, and therefore directly unit-testable.
build_roadmap(student_id) → Roadmap	M2	API layer	Returns ordered roadmap items, each carrying the gap it closes. Guarantees prerequisite ordering.
mark_stale(student_id) → None	M2	M1	Flags existing recommendations as stale. Called whenever the skill vector or target role changes.
get_resources_for_skill(skill_ids, level) → list[Resource]	M4	M2	Returns approved catalogue resources only. Falls back to external lookup or cached list when the catalogue has no match.
authorize(user, action, resource) → bool	M4	All modules	Central permission check. Every protected operation calls this; no module implements its own role logic.
record_audit(actor, action, target, reason) → None	M4	M2, M3, M4	Writes an attributable audit record for administrative and faculty actions.

6. Cross-Cutting Concerns

6.1 Security and Access Control

- Authentication uses JWT bearer tokens issued at login. Tokens carry the user id and role and are verified in API-layer middleware before any route handler executes.
- Passwords are hashed with Argon2id. No plaintext password is ever stored or logged (NFR-SEC-03).
- Authorisation is a single centralised check, M4's authorize(). No module writes its own role comparison, which prevents the common failure where one endpoint is accidentally left unguarded.
- Faculty-Mentor access is scoped by the mentor_assignment table; a faculty user cannot read a student who is not assigned to them (NFR-SEC-08).
- Uploaded files are validated for type and size before they are written to disk, and are stored outside the web root (NFR-SEC-06).
- Error responses are sanitised at the API layer so that stack traces and internal identifiers never reach the client (NFR-SEC-10).

6.2 Error Handling and Degradation

The system distinguishes three failure classes and treats them differently:

Failure class	Handling	Example
Client error	Return 4xx with a field-level, plain-language message. No retry.	Resume uploaded in an unsupported format (REQ-F1-08)
Recoverable dependency failure	Degrade to a cached or fallback path and continue serving the request.	YouTube quota exhausted — fall back to cached resource list (REQ-F2-09)
Internal error	Log with a correlation id, return a generic 500, leave stored state unchanged.	Resume parser raises unexpectedly — existing profile must be untouched (REQ-F1-38)

6.3 Caching

External resource lookups are cached per canonical skill with a configurable time to live. This is not a performance optimisation but a constraint-driven requirement: the free-tier quota (C4) would otherwise be exhausted by repeated roadmap generation for the same skills across a cohort. The cache is owned by M4 under rule R6.

6.4 Logging and Traceability

- Every request is assigned a correlation id at the API layer and that id is carried through all module calls and log lines for that request.
- Administrative and faculty actions additionally write an audit record through record_audit(), which is queryable and attributable (REQ-F4-16, BR-07).
- Skill vector changes are recorded in skill_change_history with the previous level, new level and source (REQ-F1-07), which gives a full explanation of why a student's profile says what it says.

6.5 Configuration

All environment-dependent values — database connection, JWT secret, external API key, cache TTL, confidence threshold, stalled-path interval — are supplied as environment variables and never committed. The same image therefore runs unchanged in development, CI and demonstration environments, which is what makes goal G5 achievable.

7. Technology Stack

Concern	Technology	Justification
Frontend	Next.js (React), TypeScript	Server-side rendering gives acceptable first-load performance; component model suits the four distinct console UIs; team has prior familiarity.
Backend	Python 3.12 with FastAPI	Native async support, automatic OpenAPI schema generation which keeps §5.1 honest, and Pydantic request validation at the boundary, which removes a whole class of input-validation defects from module code.
Database	PostgreSQL 16	The data model is strongly relational with meaningful foreign keys and level comparisons; transactional integrity matters for skill-vector updates. A document store would require application-side joins for gap computation.
ORM / migrations	SQLAlchemy + Alembic	Repository pattern support and versioned schema migrations, which matter when four members change the schema in parallel.
Authentication	JWT (PyJWT) + Argon2id hashing	Stateless verification avoids server-side session storage; Argon2id is the current recommended password hash.
Backend testing	pytest + pytest-cov	Mandated by C2; coverage reporting feeds the assessed unit-test coverage metric.
Frontend testing	Jest + React Testing Library	Component-level behaviour tests to complement backend unit tests.
Static analysis	SonarQube	Mandated by C2; quality gate wired into the Jenkins pipeline as a promotion blocker.
Containerisation	Docker + Docker Compose	Mandated by C2; delivers goal G5.
CI/CD	Jenkins	Mandated by C2. Pipeline: checkout → install → lint → unit tests → coverage report → SonarQube gate → docker build → deploy.
Version control	Git on GitHub, pull-request review	Mandated by C2; PR review is also the enforcement mechanism for rules R1 and R2.
External resources	YouTube Data API v3 (free tier)	Only free source with adequate coverage of technical learning content; used to enrich the catalogue, never as the system of record (G4).

8. Architecture Decision Records

Each record states the decision, the context that forced it, and the consequences the team accepts as a result.

ADR-01 — Modular monolith rather than microservices

Status	Accepted
Context	Four members each own one functionality and are individually assessed. A service-per-module split would appear to match that ownership structure.
Decision	Build a single deployable backend, internally divided into four modules with published interfaces.
Consequences	Positive: no network boundary, no distributed transactions, one CI pipeline, one deployment. Ownership is preserved through interface discipline (rules R1–R6). Negative: boundaries rest on review discipline rather than enforcement; modules cannot scale independently. Accepted because the expected load is a single cohort.

ADR-02 — Rule-based, level-aware gap matching rather than a learned model

Status	Accepted
Context	The project is framed as AI-powered. A learned recommender was considered, but no labelled dataset of student-profile-to-correct-path pairs exists, and none can be produced within the semester.
Decision	Compute the gap as a deterministic, level-aware comparison of the skill vector against the target role's required skills and levels, then rank and sequence by prerequisite order.
Consequences	Positive: outputs are deterministic and therefore unit-testable with fixed fixtures (G2); every recommendation is explainable in terms of the gap it closes (G3); no training data required. Negative: recommendation quality is bounded by the quality of the hand-maintained taxonomy and role mappings. The interface is designed so the ranking function can be replaced by a learned model later without changing M1, M3 or M4.

ADR-03 — Catalogue as system of record, external API as enrichment only

Status	Accepted
Context	One early proposal queried the YouTube Data API live on every recommendation. This conflicted with the catalogue module, which stores curated, moderated, skill-tagged resources, and with progress tracking, which needs a stable resource identity to record enrolment against.
Decision	The moderated catalogue is authoritative. The external API populates and enriches it, and acts as a live fallback only when the catalogue has no resource for a required skill.
Consequences	Positive: enrolment and progress records point at stable resources; quota consumption is bounded and cacheable; the system still works when the external service is unavailable (G4). Negative: the catalogue must be seeded with enough resources to be useful, which is a real content-entry task the team must schedule.

ADR-04 — Single writer per data domain

Status	Accepted
Context	The skill vector is written from two directions: directly by profile edits, and indirectly when a completed resource raises a skill level. With two writers, concurrent updates could silently lose a level change.
Decision	M1 is the only writer of the skill vector; M3 requests a change through <code>apply_vector_update()</code> . The same rule applies to M4 for catalogue and identity data.
Consequences	Positive: no write conflicts, one place to enforce the raise-only-if-higher rule, one place to write change history. Negative: a cross-module call on a hot path, and M3 depends on M1's availability. Accepted because both run in the same process.

ADR-05 — Event-triggered re-ranking rather than recompute-on-login

Status	Accepted
Context	Two options for refreshing recommendations after a student completes something: recompute lazily at next login, or trigger immediately on the validated change.
Decision	Mark recommendations stale at the moment the skill vector changes, and recompute on the next

Status	Accepted
	roadmap read.
Consequences	Positive: the student never sees a roadmap that contradicts work they have just completed; staleness is explicit state rather than an implicit assumption. Negative: a small extra cost on the write path. The stale flag also gives a clean recovery point if recomputation fails.

9. Requirement to Architecture Traceability

This table maps each SRS feature to the architectural elements that realise it. It is the Architecture Reference column referred to in Appendix C of the SRS.

Requirement group	Requirement IDs	Architecture element	View
Profile management and skill extraction	REQ-F1-01 to F1-24	M1 Profile Service; resume parser, skill resolver; student / profile_item / student_skill tables	§4.1, §4.2, §4.7
Skill vector computation and exposure	REQ-F1-25 to F1-30	M1 get_skill_vector(), apply_vector_update(); HAS_SKILL relationship	§5.2, §4.6
Target role selection	REQ-F1-31 to F1-34	M1 target-role endpoint; CAREER_ROLE and role_skill_requirement	§5.1, §4.6
Gap computation and ranking	REQ-F2-01 to F2-04	M2 compute_gap(), rank_gap() — pure functions, no I/O	§4.2, §5.2
Roadmap generation and sequencing	REQ-F2-05 to F2-08	M2 build_roadmap(), sequence_prereqs(); skill_prerequisite edges	§4.5, §5.2
External fallback and caching	REQ-F2-09, F2-10	M4 get_resources_for_skill() with per-skill cache; ADR-03	§6.3, §8
Enrolment and progress recording	REQ-F3-01 to F3-04	M3 Progress Service; progress table	§4.2, §4.7
Completion evidence and validation	REQ-F3-05 to F3-07	M3 evidence validator; completion_evidence table	§4.7, §6.2
Adaptive skill-vector update and re-rank	REQ-F3-08, F3-09, F1-29, F1-30	M3 → M1 apply_vector_update() → M2 mark_stale(); ADR-04, ADR-05	§4.5, §8
Dashboards, milestones, reminders	REQ-F3-10 to F3-14	M3 dashboard and notifier; milestone and notification tables	§4.2, §4.7
Authentication and role assignment	REQ-F4-01, F4-02	M4 auth routes; JWT middleware; user_account, role tables	§6.1, §5.1
Authorisation enforcement	REQ-F4-03, F4-04, F4-18, F1-39, F1-40	M4 authorize() — single centralised check called by all modules	§5.2, §6.1
Catalogue management and moderation	REQ-F4-05 to F4-12	M4 Catalogue Service; learning_item with moderation_state	§4.2, §4.7
Faculty views, endorse and override	REQ-F4-13 to F4-15	M4 faculty routes; mentor_assignment; recommendation override records	§5.1, §4.7
Audit	REQ-F4-16, BR-07	M4 record_audit(); audit_record table	§6.4, §4.7
Performance requirements	NFR-PERF-01 to 05	Pagination at API layer; per-skill cache; pure-function gap computation	§6.3, §3.1
Security requirements	NFR-SEC-01 to 10	JWT middleware, Argon2id, centralised authorize(), sanitised errors, network isolation	§6.1, §4.4
Reproducible deployment	Constraint C2, Goal G5	docker-compose definition; Jenkins pipeline with SonarQube gate	§4.4, §7

10. Risks and Open Architectural Issues

#	Risk or open issue	Impact	Mitigation / action
A1	Skill taxonomy and role-requirement data must be hand-seeded before anything demonstrable exists. This is unglamorous work with no owner in the module split.	High — without seed data, gap computation returns nothing and the system cannot be demonstrated at all.	Treat taxonomy seeding as a shared backlog item in Sprint 1, split across all four members rather than left to M4.
A2	Module boundaries are enforced by review, not by tooling. A cross-module internal import will run correctly and go unnoticed.	Medium — accumulates hidden coupling that surfaces late.	Directory structure mirrors module boundaries; add an import-linting rule to the CI pipeline if violations appear.
A3	Resume parsing accuracy on unstructured PDFs is the least predictable part of the system.	Medium — poor extraction degrades every downstream recommendation.	Confirmation step (REQ-F1-15) means no extracted skill enters the profile without student review, so accuracy failures degrade convenience rather than correctness.
A4	Unit-testing the CI pipeline against external API calls will be flaky and will consume quota.	Medium — flaky CI erodes trust in the pipeline.	The external client is behind M4's interface, so it is mocked in all tests. No test hits the live API.
A5	ADR-05 leaves recomputation on the read path. A burst of completions immediately before many roadmap reads could produce a visible latency spike.	Low at expected cohort load.	Accepted for Phase 1. Revisit if NFR-PERF-03 is missed in system testing.